

	DEPARTAMENT D'INFORMÀTICA		CURS 2025-2026
	2N DAW	M0616	Projecte
	Oriol Torra Tudela		Data: 22/05/2026

M0616-Projecte

ChessHub – Memòria del projecte

Versió 1.0

Data d'actualització: 22 de maig de 2026



Índex

1. Elements preliminars.....	1
1.1. Portada.....	1
1.2. Resum executiu.....	1
2. Cos principal.....	3
2.1. Introducció.....	3
2.2. Planificació.....	5
2.3. Desenvolupament del projecte.....	8
2.4. Avaluació i resultats.....	16
3. Elements finals.....	19
3.1. Conclusions.....	19
3.2. Referències i bibliografia.....	21
3.3. Annexos.....	22

1. Elements preliminars

1.1. Portada

La portada d'aquesta memòria és la pàgina anterior. Hi consten el títol del projecte (ChessHub), el nom de l'autor, el centre i el cicle formatiu, el curs acadèmic, la versió del document i la data d'elaboració.

1.2. Resum executiu

ChessHub és una plataforma web per jugar a escacs en línia, desenvolupada com a Projecte de Síntesi del cicle formatiu de grau superior de Desenvolupament d'Aplicacions Web. L'objectiu principal del projecte ha estat dissenyar, construir i desplegar una aplicació completa i funcional que permeti jugar a escacs des de qualsevol navegador, sense necessitat d'instal·lar res, i que serveixi alhora com a producte real i com a exercici tècnic que integri els coneixements adquirits durant tot el cicle.

L'aplicació ofereix partides multijugador en temps real entre usuaris registrats, partides d'entrenament contra un motor d'intel·ligència artificial, una col·lecció de problemes tàctics, un sistema social d'amics i missatgeria, una classificació global per puntuació ELO i el seguiment de l'historial de partides. Tot plegat s'ha construït amb una arquitectura de tres capes desacoblades, amb un servei addicional de temps real, i s'ha desplegat en un entorn de producció real sobre un clúster Kubernetes.

Els objectius plantejats a l'inici del projecte s'han assolit íntegrament: l'aplicació és operativa, està desplegada i disponible públicament, i incorpora dos elements innovadors destacats, la comunicació en temps real entre jugadors i un motor d'escacs professional executat dins el mateix navegador. A més de les funcionalitats previstes, s'han incorporat diverses ampliacions que enriqueixen el producte final.

Resultats clau aconseguits:

- Aplicació web completa, desplegada i funcional en un entorn de producció real (el clúster Kubernetes del centre).
- Arquitectura de tres capes: aplicació de client amb Angular, API REST amb CodeIgniter i servei de temps real amb Node.js i Socket.IO.
- Model de dades relacional de quinze taules sobre MariaDB, amb Redis com a memòria cau.
- Integració del motor Stockfish compilat a WebAssembly, que aporta una intel·ligència artificial real executada al navegador del client.
- Suite de proves automatitzades i revisió de seguretat del codi, amb la documentació tècnica i d'usuari corresponent.

2. Cos principal

2.1. Introducció

Context del projecte

Aquest projecte s'emmarca dins el mòdul de Projecte de Síntesi del segon curs del cicle formatiu de grau superior de Desenvolupament d'Aplicacions Web. La finalitat d'aquest mòdul és integrar, en un únic treball de certa envergadura, els coneixements adquirits al llarg de tot el cicle: la programació, el disseny i la gestió de bases de dades, el desenvolupament d'interfícies, el desplegament d'aplicacions i el treball en entorns reals de producció.

Els escacs són un joc amb una comunitat enorme i en creixement constant, i les plataformes en línia n'han popularitzat encara més la pràctica durant els darrers anys. Partint d'aquest interès, es va proposar construir una plataforma d'escacs pròpia que fos completa i funcional. La tria d'aquest tema responia a un doble criteri: d'una banda, oferir un producte atractiu i útil; de l'altra, plantejar un repte tècnic ampli, ja que una aplicació d'escacs en línia obliga a combinar interfície d'usuari, lògica de servidor, comunicació en temps real, persistència de dades i desplegament, és a dir, pràcticament tots els àmbits treballats al cicle.

Objectius detallats

El projecte es va plantejar amb un objectiu general desenvolupar i desplegar una plataforma d'escacs en línia completa que es concreta en els objectius específics següents:

- Permetre el registre i l'autenticació segura dels usuaris de la plataforma.
- Oferir partides d'escacs multijugador en temps real, amb validació automàtica de les regles i control de temps.
- Incorporar un adversari controlat per ordinador amb diversos nivells de dificultat.
- Proporcionar una col·lecció de problemes tàctics per a l'entrenament dels usuaris.
- Implementar un sistema de puntuació ELO i una classificació global de jugadors.

- Desenvolupar funcions socials: sol·licituds d'amistat, llista d'amics i missatgeria privada.
- Aconseguir una interfície clara, usable i adaptable a diferents mides de pantalla.
- Desplegar l'aplicació en un entorn de producció real i mantenir-la operativa i accessible.

Abast del projecte

L'abast del projecte inclou el desenvolupament complet de l'aplicació en totes les seves parts: la capa visible al navegador, l'API de servidor i el servei de temps real. Inclou també el disseny i la implementació de la base de dades, el desplegament de tot el conjunt en un entorn de producció real i l'elaboració de la documentació tècnica i d'usuari que acompanya el producte.

Queden fora de l'abast del projecte el desenvolupament d'una aplicació mòbil nativa tot i que la interfície web s'ha dissenyat de manera adaptable i és plenament utilitzable des de mòbils i tauletes, la integració de passarel·les de pagament i l'enviament de correus electrònics. Aquesta darrera funció es va descartar durant el desenvolupament per limitacions de l'entorn, tal com es detalla a l'apartat de desviacions de la planificació.

Metodologia utilitzada

S'ha seguit una metodologia de desenvolupament iterativa i incremental. El treball es va organitzar en fases successives i, a cada fase, s'afegia un conjunt de funcionalitats que es provaven i es desplegaven abans de continuar amb les següents. D'aquesta manera, l'aplicació es va mantenir en un estat funcional i desplegable durant tot el procés, i cada nova funcionalitat es construïa sobre una base ja verificada.

El control de versions s'ha gestionat amb Git, amb un historial de canvis complet que ha permès treballar de manera ordenada i revisar l'evolució del projecte. Les sessions setmanals de classe han servit com a punt de seguiment: a cadascuna es revisava el progrés assolit, es detectaven les desviacions respecte de la planificació i s'ajustaven les prioritats per a les setmanes següents. Les proves de cada funcionalitat es feien a mesura que es desenvolupava, i no pas al final, per detectar els problemes com més aviat millor.

2.2. Planificació

Pla de treball general i fases del projecte

El projecte es va dividir en sis fases consecutives, des de la proposta inicial fins al tancament i la documentació. Aquesta divisió respon a un ordre lògic de dependències: no es pot desenvolupar sense haver definit abans l'arquitectura, i no es poden afegir funcionalitats avançades sense disposar d'un nucli estable. La taula següent en resumeix el calendari i les activitats principals; es tracta d'un cronograma propi, elaborat a partir de les dates clau del mòdul i de l'historial real de desenvolupament.

Fase	Període	Activitats principals
1. Proposta i planificació	08/03 – 15/03	Proposta del projecte, aprovació i lliurament de la planificació.
2. Anàlisi i disseny	16/03 – 29/03	Definició de requisits, model de dades, arquitectura i elecció de l'stack tecnològic.
3. Configuració entorn	30/03 – 12/04	Repositori, contenidors, esquelet de les tres capes i creació de la base de dades.
4. Desenvolupament del nucli	13/04 – 03/05	Autenticació, partides en temps real i taulell de joc. Lliurament parcial.
5. Funcionalitats i integració	04/05 – 15/05	Motor d'IA, problemes, social, rànquing, anàlisi, proves i revisió de seguretat.
6. Documentació	16/05 – 22/05	Manuals, memòria, proves finals i lliurament final del projecte.

Cronograma: dates clau

La planificació del projecte ha estat condicionada per les dates clau del mòdul, que marquen els lliuraments i les fites obligatòries. Són les següents:

Data	Fita
08-03-2026	Lliurament de les propostes del projecte intermodular.
09-03-2026	Publicació de la llista de propostes admeses.
15-03-2026	Lliurament de la planificació.
16-03-2026	Inici del projecte intermodular i de les classes de suport.
11-04-2026	Inici de les classes en hores del mòdul M613.
27-04-2026	Fi de les classes de suport; les hores passen a desenvolupament del projecte.
03-05-2026	Lliurament parcial.
22-05-2026	Lliurament final.
25/27-05-2026	Presentació dels projectes.

Recursos necessaris

Els recursos emprats per dur a terme el projecte han estat els següents:

- **Recursos humans:** una sola persona, l'autor del projecte (jo, Oriol Torra), amb el suport, el seguiment i l'assessorament del professorat del centre.
- **Maquinari:** un ordinador personal per al desenvolupament i el servidor d'aplicacions del centre, format per un clúster Kubernetes, on s'ha desplegat l'aplicació.
- **Programari:** un editor de codi, Git per al control de versions, Docker per als contenidors, i els entorns d'execució de Node.js i PHP, a més dels navegadors web. Totes les eines tenen llicència lliure o gratuïta.
- **Infraestructura:** un repositori de codi a GitHub, un registre d'imatges de contenidors (Harbor) i el clúster Kubernetes, aquests dos darrers proporcionats pel centre.

Pla de treball detallat i tasques setmanals

Durant les primeres setmanes (mes de març) el treball es va concentrar en les tasques d'anàlisi i disseny: la definició dels requisits funcionals, el disseny del model de dades relacional i l'elecció de les tecnologies de cada capa. Tot seguit es va preparar l'entorn de treball, amb la creació del repositori, la configuració dels contenidors i la posada en marxa de l'esquelet de les tres capes de l'aplicació.

El mes d'abril es va dedicar al desenvolupament del nucli de l'aplicació. Primer es va implementar el sistema d'autenticació i, a continuació, el taulell de joc i les partides multijugador en temps real, que constitueixen la funcionalitat central. Aquesta part va requerir resoldre la sincronització entre els dos jugadors i el control del temps. Després, a principis de maig, l'esforç es va orientar a les funcionalitats addicionals partides contra la màquina, problemes, sistema social, classificació i anàlisi i, posteriorment, a les tasques de qualitat: les proves automatitzades i la revisió de seguretat. Les darreres setmanes de maig s'han reservat per a l'elaboració de la documentació i les proves finals del conjunt.

Fites assolides

Al llarg del projecte s'han anat assolint les fites previstes dins els terminis establerts:

- Lliurament de la planificació dins el termini (15 de març).
- Definició de l'arquitectura del sistema i preparació de l'entorn de desenvolupament.
- Primer desplegament funcional de l'aplicació al clúster Kubernetes del centre.
- Partides multijugador en temps real plenament operatives.
- Integració del motor Stockfish i finalització de totes les funcionalitats previstes.
- Suite de proves automatitzades i revisió de seguretat del codi completades.
- Documentació tècnica i d'usuari enllestida abans del lliurament final.

Desviacions de la planificació

El projecte s'ha mantingut, en conjunt, dins el calendari previst, i tots els lliuraments s'han fet en termini. Tot i així, s'han produït algunes desviacions respecte de la previsió inicial. La més significativa va ser el replantejament del mòdul d'intel·ligència artificial: la previsió inicial es basava en l'ús d'un servei extern per generar les jugades de l'adversari, però aquesta solució es va substituir per la integració d'un motor d'escacs executat al mateix navegador, una opció més robusta que va requerir feina addicional no prevista.

També es van ampliar les funcions socials respecte de la previsió inicial, amb la incorporació de la missatgeria privada entre amics, i es va descartar l'enviament de correus electrònics per limitacions de l'entorn. Cap d'aquestes desviacions ha posat en risc els lliuraments ni ha alterat substancialment el calendari; al contrari, han contribuït a millorar el resultat final del producte.

2.3. Desenvolupament del projecte

Descripció de la solució implementada

ChessHub és una aplicació web que s'executa íntegrament al navegador de l'usuari i que es comunica amb un conjunt de serveis allotjats al servidor. L'usuari accedeix a l'aplicació a través d'una adreça web, es registra o inicia sessió i, a partir d'aquell moment, té a la seva disposició totes les funcionalitats de la plataforma: pot jugar partides contra altres persones o contra la màquina, resoldre problemes, consultar la seva evolució i relacionar-se amb altres jugadors.

La solució no és un únic programa monolític, sinó un conjunt de serveis independents que cooperen entre si. Cada servei té una responsabilitat ben definida, servir la interfície, gestionar la lògica de negoci, sincronitzar les partides en temps real o emmagatzemar les dades i es comunica amb la resta mitjançant protocols estàndard. Aquesta separació facilita el desenvolupament, les proves i el manteniment, i permet desplegar i actualitzar cada part de manera independent.

Tots els serveis s'empaqueten en contenidors i s'executen en un clúster Kubernetes, que els orquestra i en garanteix la disponibilitat. El resultat és una aplicació desplegada en un entorn de producció real, accessible públicament des de qualsevol navegador.

El codi del projecte s'organitza en un únic repositori que conté, en directoris separats, les tres parts del sistema: l'aplicació de client, l'API de servidor i el servei de temps real. Aquesta organització manté tot el projecte unificat i n'afavoreix la coherència, alhora que cada part conserva la seva pròpia configuració i es pot construir i desplegar de manera autònoma. L'entorn de desenvolupament reproduïx amb contenidors locals la mateixa estructura de serveis que l'entorn de producció, cosa que redueix les diferències entre tots dos i evita errors derivats de la configuració.

Des del punt de vista de l'usuari, ChessHub es presenta com una aplicació única i fluida. En tractar-se d'una aplicació d'una sola pàgina, el pas d'una secció a una altra, jugar, problemes, classificació, perfil o amics és immediat i no comporta esperes ni recàrregues completes de la pàgina. A més, la interfície s'ha dissenyat de manera adaptable: el mateix codi s'ajusta automàticament a la mida de la pantalla i ofereix una experiència coherent tant en un ordinador de sobretaula com en una tauleta o un telèfon mòbil.

Tecnologies utilitzades

Per construir la solució s'han triat tecnologies actuals, madures i àmpliament utilitzades a la indústria del desenvolupament web. La taula següent les resumeix per capes, i a continuació se'n justifica l'elecció.

Capa	Tecnologies principals
Aplicació de client	Angular (TypeScript), Bootstrap, chess.js i el motor Stockfish.
API de servidor	CodeIgniter 4 sobre PHP, amb autenticació mitjançant JWT.
Servei de temps real	Node.js amb les biblioteques Socket.IO i Express.
Capa de dades	MariaDB com a base de dades relacional i Redis com a memòria cau.
Infraestructura	Docker, Kubernetes, el registre d'imatges Harbor i el servidor web nginx.
Eines de treball	Git i GitHub, Visual Studio Code, i PHPUnit i Vitest per a les proves.

La capa de presentació s'ha desenvolupat amb Angular, un framework de codi obert per construir aplicacions web. S'ha triat per la seva maduresa, per la seva àmplia adopció a la indústria i per l'ús del llenguatge TypeScript, que afegeix un sistema de tipus sobre JavaScript i ajuda a detectar molts errors abans d'executar el codi. L'aplicació s'organitza en components reutilitzables i les seves seccions es carreguen de manera mandrosa, és a dir, només quan es necessiten, cosa que millora el temps de càrrega inicial. El resultat és una aplicació d'una sola pàgina, en què la navegació entre seccions és fluida i no requereix recarregar la pàgina sencera.

Sobre aquesta base, dues tecnologies complementen la capa de client. La biblioteca chess.js s'encarrega de la lògica dels escacs: valida que cada jugada sigui legal, detecta les situacions d'escac i mat i de taules, i gestiona els casos especials com l'enroc o la promoció de peons. El conjunt d'estils Bootstrap, per la seva banda, aporta una base visual coherent i, sobretot, un sistema de disseny adaptable que permet que la interfície es vegi correctament tant en un ordinador com en una tauleta o un telèfon mòbil.

L'adversari controlat per ordinador es basa en Stockfish, un dels motors d'escacs més potents i reconeguts del món, distribuït amb llicència lliure. El motor s'ha compilat al format WebAssembly, una tecnologia que permet executar codi d'alt rendiment directament dins el navegador, a una velocitat propera a la d'un programa natiu. A més, el motor s'executa en un fil d'execució separat perquè els seus càlculs no bloquegin la interfície i l'usuari pugui continuar interactuant amb l'aplicació mentre la màquina pensa.

La capa de negoci és una API REST desenvolupada amb CodeIgniter 4, un framework lleuger del llenguatge PHP. Aquesta API concentra tota la lògica de l'aplicació, la gestió d'usuaris i partides, el càlcul de la puntuació ELO, la validació de les dades i és l'únic component que accedeix directament a la base de dades. El codi s'organitza en controladors, que reben les peticions, i en models, que representen les dades. L'autenticació es resol amb testimonis JWT, que viatgen amb cada petició i permeten identificar l'usuari sense que el servidor hagi de mantenir cap sessió oberta.

La comunicació en temps real es gestiona amb un servei independent desenvolupat amb Node.js, l'entorn d'execució de JavaScript al servidor, i les biblioteques Socket.IO i Express. Aquest servei manté connexions WebSocket obertes amb tots els clients connectats i s'encarrega de sincronitzar les partides en directe, el xat i els reptes entre jugadors. S'ha decidit separar-lo de l'API REST perquè les seves necessitats són molt diferents: mentre que l'API atén peticions puntuals i independents, el servei de temps real ha de mantenir connexions persistents i enviar dades de manera contínua.

Les dades persistents de l'aplicació es desen en una base de dades relacional MariaDB, un sistema de gestió de bases de dades de codi obert i àmpliament utilitzat. La informació s'estructura en quinze taules relacionades entre si. A més, s'utilitza Redis, un magatzem de dades en memòria molt ràpid, com a memòria cau: s'hi guarden temporalment els resultats de les operacions que es repeteixen sovint, com l'anàlisi de posicions, de manera que es poden tornar a oferir sense haver de recalcular-los ni consultar de nou la base de dades.

Finalment, pel que fa a la infraestructura, tots els serveis del sistema s'empaqueten en contenidors amb Docker. Un contenidor és una unitat de programari aïllada que inclou tot el que un servei necessita per funcionar, fet que garanteix que s'executi igual en qualsevol entorn. Aquests contenidors es despleguen sobre un clúster Kubernetes, que els orquestra, els manté en funcionament i els reinicia si cal. Les imatges dels contenidors es publiquen en un registre Harbor i un servidor web nginx s'encarrega de servir l'aplicació de client als usuaris.

Pel que fa a les eines de treball, el codi s'ha escrit amb l'editor Windsurf i s'ha versionat amb Git, amb el repositori allotjat a GitHub. El procés de construcció i desplegament parteix sempre del codi del repositori: a partir

d'aquest es generen les imatges dels contenidors de cada servei, es publiquen al registre Harbor i, finalment, s'apliquen al clúster Kubernetes, que substitueix els serveis en funcionament per la versió nova. Aquest flux fa que publicar una actualització sigui un procés ordenat, controlat i repetible.

Arquitectura de la solució

ChessHub segueix una arquitectura client-servidor de tres capes, amb una separació estricta de responsabilitats, a la qual s'afegeix un servei especialitzat de comunicació en temps real. Aquesta organització és un dels patrons més habituals i provats en el desenvolupament d'aplicacions web, i facilita tant el manteniment com l'escalabilitat.

La capa de presentació és l'aplicació de client, escrita amb Angular, que s'executa al navegador de l'usuari. És responsable de mostrar la interfície, recollir les accions de l'usuari i comunicar-se amb el servidor. La capa de negoci és l'API REST, que conté la lògica de l'aplicació, validacions, càlcul de la puntuació ELO, gestió de partides i usuaris i és l'única que accedeix directament a les dades. La capa de dades la formen la base de dades relacional, que emmagatzema tota la informació persistent, i la memòria cau Redis, que guarda temporalment dades d'accés freqüent.

A aquestes tres capes s'hi afegeix el servei de temps real. Mentre que l'API REST funciona amb peticions puntuals (el client pregunta i el servidor respon), una partida d'escacs en directe necessita que els dos jugadors rebin els canvis a l'instant, sense haver de preguntar contínuament. Per això, el servei de temps real manté un canal de comunicació obert amb cada client mitjançant la tecnologia WebSocket i hi envia les actualitzacions tan bon punt es produeixen.

El motor d'intel·ligència artificial mereix una menció a part dins l'arquitectura, perquè no s'executa al servidor sinó dins el navegador de cada usuari. Aquesta decisió, descrita amb més detall a l'apartat següent, va ser determinant: el servidor del centre no pot allotjar models d'intel·ligència artificial, de manera que situar el motor al client va ser l'opció que va fer viable aquesta funcionalitat.

Per il·lustrar el funcionament conjunt, val la pena seguir el recorregut d'una jugada en una partida en línia. Quan un jugador mou una peça, l'aplicació de client valida primer que la jugada sigui legal segons les regles dels escacs. Tot seguit, la jugada es transmet pel servei de temps real, que la fa arribar a

l'instant a l'adversari, i es desa de manera permanent a la base de dades a través de l'API. El rellotge de la partida el controla sempre el servidor, que actua com a única font de veritat del temps; quan la partida acaba, el resultat i la nova puntuació ELO de tots dos jugadors es desen de manera atòmica.

El recorregut d'una jugada en una partida contra la màquina segueix un camí diferent. Quan l'usuari mou una peça, l'aplicació de client valida la jugada i, tot seguit, demana al motor Stockfish que calculi la millor resposta possible. El motor analitza la posició i, al cap d'uns segons, retorna la seva jugada, que s'aplica automàticament al taulell. Tant la jugada de l'usuari com la del motor es desen a la base de dades a través de l'API. En aquesta modalitat no intervé el servei de temps real, ja que no hi ha cap segon jugador humà amb qui calgui sincronitzar la partida.

La capa de dades s'ha dissenyat seguint el model relacional. La informació es reparteix en quinze taules, cadascuna dedicada a una entitat concreta del sistema, usuaris, perfils, partides, jugades, problemes, amistats, missatges, entre d'altres, i les taules es relacionen entre si mitjançant claus que garanteixen la integritat de les dades. Aquest disseny evita la duplicació d'informació i permet fer consultes complexes de manera eficient, com ara recuperar l'historial complet de partides d'un usuari o construir la classificació global ordenada per puntuació.

Pel que fa al desplegament, cada servei del sistema s'executa dins el clúster Kubernetes en forma de contenidors. Kubernetes s'encarrega de mantenir tots els serveis en funcionament, de reiniciar-los automàticament si algun falla i de gestionar les peticions que els arriben. L'aplicació queda exposada a l'exterior a través d'una adreça web pública, de manera que qualsevol usuari hi pot accedir des del navegador. Aquesta infraestructura, malgrat la seva complexitat, aporta robustesa al sistema i en facilita les actualitzacions: per publicar una versió nova n'hi ha prou de generar les imatges actualitzades dels contenidors i indicar al clúster que les apliqui.

La comunicació entre l'aplicació de client i l'API segueix l'estil REST. L'API exposa els seus serveis com un conjunt de recursos, usuaris, partides, problemes, amics, entre d'altres, als quals s'accedeix mitjançant les operacions estàndard del protocol HTTP. Aquesta organització fa que la interfície entre el client i el servidor sigui clara i previsible, i facilita tant el desenvolupament com la depuració d'errors.

La seguretat s'ha tingut en compte des del disseny mateix de l'arquitectura. Les contrasenyes dels usuaris no es desen mai en text pla, sinó xifrades amb un algorisme pensat específicament per a aquesta finalitat. L'accés als serveis protegits es controla amb els testimonis JWT: en iniciar sessió, l'usuari rep un testimoni d'accés de durada curta i un testimoni de renovació de durada més llarga, que permet obtenir-ne un de nou sense tornar a introduir les credencials. Cada petició a una zona protegida ha d'anar acompanyada d'un testimoni vàlid, que el servidor comprova abans de respondre.

El paper de la memòria cau es pot il·lustrar amb l'anàlisi de partides. Analitzar una posició d'escacs és una operació costosa en temps de càlcul; per evitar repetir-la, un cop calculada, el resultat es desa a Redis durant un període determinat. Si més endavant es torna a sol·licitar l'anàlisi de la mateixa posició, el sistema la recupera directament de la memòria cau, de manera gairebé instantània i sense haver de carregar la base de dades.

Funcionalitats principals

Autenticació d'usuaris. ChessHub disposa d'un sistema de registre i autenticació. En registrar-se, cada persona crea un compte protegit amb una contrasenya, que es desa xifrada i mai en text pla, de manera que ni tan sols els administradors hi tenen accés directe. L'accés posterior es fa mitjançant l'inici de sessió amb el nom d'usuari i la contrasenya. Un cop dins, la identitat de l'usuari es manté durant tota la navegació gràcies a un testimoni de seguretat que acompanya cada petició al servidor i que caduca al cap d'un temps per raons de seguretat.

Partides en línia. Les partides multijugador en temps real són la funcionalitat central i més complexa de la plataforma. Dos usuaris s'enfronten en una partida d'escacs en què totes les regles, moviments legals, escac, escac i mat, enroc, promoció i taules, es validen automàticament. Cada jugador disposa d'un rellotge, i el temps el controla el servidor per garantir-ne la coherència. Tots dos jugadors veuen el taulell actualitzat a l'instant i poden intercanviar missatges mitjançant un xat integrat. En finalitzar, el resultat de la partida modifica la puntuació ELO dels dos jugadors i la partida es desa a l'historial.

Partides contra la intel·ligència artificial. L'aplicació permet jugar partides d'entrenament contra el motor d'escacs Stockfish. Abans de començar, l'usuari pot triar entre deu nivells de dificultat, des d'un de molt assequible per a principiants fins a un de molt exigent i escollir el color de les seves peces. El motor s'executa dins el mateix navegador i analitza la posició per respondre amb una jugada de qualitat en pocs segons. Aquesta modalitat és ideal per practicar sense pressió i per preparar-se abans de jugar partides classificades.

Problemes tàctics. El mòdul de puzzles ofereix una col·lecció de posicions d'escacs reals que l'usuari ha de resoldre trobant la millor jugada o seqüència de jugades. És una eina d'entrenament orientada a desenvolupar la visió tàctica i la capacitat de càlcul, dues habilitats clau per millorar el joc. L'aplicació indica a l'usuari si la jugada proposada és correcta, i cada intent queda registrat per permetre el seguiment del progrés al llarg del temps.

Perfil i estadístiques. Cada usuari disposa d'un perfil personal que actua com a resum de la seva activitat a la plataforma. Hi consten la puntuació ELO actual, el nombre de partides guanyades, perdudes i acabades en taules, i l'evolució històrica de la puntuació al llarg del temps. Des del perfil, a més, l'usuari pot personalitzar l'aspecte visual del taulell escollint entre diversos temes. D'aquesta manera, el perfil ofereix alhora una visió del rendiment del jugador i un espai de personalització.

Classificació global. La classificació ordena tots els jugadors de la plataforma segons la seva puntuació ELO, de la més alta a la més baixa. Permet veure d'un cop d'ull qui són els jugadors més forts i situar el nivell propi en relació amb la resta de la comunitat. La classificació s'actualitza de manera automàtica cada cop que es disputa una partida classificada, de manera que sempre reflecteix l'estat real de la competició.

Amics i missatgeria. El sistema social de ChessHub permet als usuaris connectar entre si. Un usuari pot cercar altres jugadors pel seu nom, enviar-los una sol·licitud d'amistat i, si l'accepten, afegir-los a la seva llista d'amics. Entre amics es poden intercanviar missatges privats a través d'un xat. Aquesta funcionalitat fa la plataforma més propera i facilita organitzar partides amb persones conegudes.

Historial de partides. Totes les partides que un usuari juga, tant en línia com contra la màquina, queden desades de manera permanent al seu historial.

Administració. Finalment, l'aplicació inclou un panell d'administració, accessible únicament per als comptes que tenen el perfil d'administrador. Des d'aquest panell es poden gestionar els informes i les denúncies generats pels usuaris i supervisar el funcionament general de la plataforma. Aquesta separació de privilegis garanteix que les tasques de gestió quedin reservades a les persones autoritzades.

Decisions tècniques rellevants

- Arquitectura desacoblada en serveis.** Separar el client, l'API i el servei de temps real en components independents facilita el desenvolupament i el manteniment, i permet desplegar i escalar cada part per separat.

- Autenticació sense estat amb JWT.** El servidor no guarda sessions; cada petició s'autentica amb un testimoni signat. Aquesta opció simplifica l'escalabilitat i s'adapta bé a una arquitectura amb diversos serveis.

- Comunicació en temps real amb WebSocket.** L'ús de Socket.IO permet que les partides es vegin sincronitzades a l'instant entre tots dos jugadors, amb el rellotge controlat sempre pel servidor per garantir-ne la coherència.

- Motor d'IA al navegador.** Stockfish, compilat a WebAssembly, s'executa al client. Aquesta decisió va ser clau, ja que el servidor del centre no pot allotjar models d'intel·ligència artificial; situar el motor al navegador va fer viable la funcionalitat sense consumir recursos de servidor.

- Base de dades relacional.** S'ha optat per una base de dades relacional perquè la informació del projecte, usuaris, partides, jugades, relacions d'amistat, està molt estructurada i interconnectada, un escenari en què el model relacional aporta integritat i consultes potents.

- Validació de les jugades al client.** La legalitat de cada jugada es comprova al navegador amb una biblioteca especialitzada, cosa que dona una resposta immediata a l'usuari i evita processar jugades incorrectes.

- Operacions de base de dades atòmiques.** Les accions sensibles a la concurrència, unir-se a una partida, finalitzar-la o actualitzar la puntuació, es resolen amb operacions atòmiques que eviten les condicions de cursa.

- Memòria cau amb Redis.** Les operacions repetitives, com l'anàlisi de posicions, es guarden temporalment a Redis per respondre més de pressa i alleugerir la càrrega de la base de dades.

- Contenidors i orquestració.** L'ús de Docker i Kubernetes garanteix un desplegament reproduïble, fàcil de mantenir i coherent entre els diferents entorns.

- Disseny adaptable.** La interfície s'ha construït perquè s'ajusti automàticament a qualsevol mida de pantalla, de manera que l'aplicació és plenament utilitzable tant des d'un ordinador com des d'un dispositiu mòbil, sense necessitat d'una versió separada.

La part innovadora del projecte rau en la combinació de dues tecnologies: la comunicació en temps real amb Socket.IO per a les partides multijugador i un motor d'escacs professional, Stockfish, executat dins el navegador mitjançant WebAssembly. Aquesta segona decisió aporta una intel·ligència artificial real i de qualitat sense consumir cap recurs de servidor, fet que la fa especialment adequada per a l'entorn del projecte.

2.4. Avaluació i resultats

Anàlisi del grau d'assoliment dels objectius

La valoració global del projecte és molt positiva. Tots els objectius plantejats a l'inici, tant l'objectiu general com els objectius específics, s'han assolit. L'aplicació és plenament funcional, està desplegada en un entorn de producció real i és accessible públicament des de qualsevol navegador. A més, incorpora totes les funcionalitats previstes i diverses ampliacions afegides durant el desenvolupament.

Avaluació per punts

Objectius assolits. S'ha completat la totalitat dels objectius descrits a la introducció. L'aplicació cobreix el cicle d'ús complet, des del registre d'un usuari nou fins a la revisió de les partides passades, passant per totes les modalitats de joc previstes. No ha quedat cap objectiu inicial sense desenvolupar, i cadascun s'ha portat fins a un estat plenament funcional, i no merament esbossat.

Ampliacions realitzades. A més del que estava previst inicialment, s'han incorporat diverses funcionalitats addicionals que enriqueixen el producte final: la missatgeria privada entre amics, l'anàlisi de les partides un cop acabades, un panell d'administració, un mode espectador que permet seguir les partides d'altres jugadors, diverses variants de control de temps i la personalització visual del taulell. Aquestes ampliacions no formaven part del compromís inicial, sinó que van sorgir de la valoració que, un cop assolit el nucli de l'aplicació, hi havia marge i temps per millorar l'experiència de l'usuari.

Grau d'implementació. El grau d'implementació del projecte és total: el cent per cent de les funcionalitats previstes estan desenvolupades, provades i desplegades a l'entorn de producció. L'aplicació no es troba en estat de prototip ni de versió parcial, sinó que constitueix un producte complet i operatiu, que es podria posar a disposició d'usuaris reals sense canvis substancials.

Eines utilitzades. El desenvolupament s'ha dut a terme amb Visual Studio Code com a editor de codi, i amb Git i GitHub per al control de versions i l'allotjament del repositori. Per a la contenidorització i el desplegament s'han fet servir Docker i Kubernetes, i per a les proves automatitzades, les eines PHPUnit, a la part de servidor, i Vitest, a la part de client. Durant el desenvolupament, l'API també s'ha provat manualment amb el navegador i amb eines de prova de peticions HTTP.

Qualitat i proves

Per assegurar la qualitat i la fiabilitat del codi s'han fet un seguit de proves automatitzades. Aquestes proves verifiquen, de manera repetible, que els components clau de l'aplicació es comporten com s'espera, i permeten detectar ràpidament si un canvi posterior n'ha trencat el funcionament.

En el servidor, amb PHPUnit, s'han provat components crítics com la generació i la validació dels testimonis d'autenticació, comprovant que es rebutgen correctament els testimonis manipulats, caducats o signats amb una clau incorrecta. En el client, amb Vitest, s'han provat serveis clau de l'aplicació. A més de les proves automatitzades, s'ha dut a terme una revisió de seguretat del codi que va permetre detectar i corregir punts febles, com la presència de credencials escrites directament al repositori.

Incidències i resolucions

Incidència 1: Adversari per ordinador. La primera versió de l'adversari per ordinador depenia d'un servei extern per obtenir les jugades i recorria a una jugada aleatòria quan aquest servei fallava, cosa que no constituïa una intel·ligència artificial real. Es va resoldre integrant el motor Stockfish, compilat a WebAssembly, que s'executa al navegador. La lliçó apresada és que cal prioritzar les solucions que no depenguin de serveis externs sobre els quals no es té control.

Incidència 2: Pes del motor d'IA. La versió més recent del motor d'escacs, basada en una xarxa neuronal, pesava massa per carregar-se còmodament al navegador. Es va resoldre fent servir una versió clàssica del motor, molt més lleugera i prou potent per a l'ús previst. La lliçó apresada és la necessitat d'equilibrar la potència d'una solució amb el seu rendiment de càrrega.

Incidència 3: Sincronització del rellotge. En les partides en temps real, el temps mostrat al client i al servidor es podia desincronitzar i provocar discrepàncies. Es va resoldre fent que el servidor fos l'única autoritat sobre el temps i enviés actualitzacions periòdiques als jugadors. La lliçó apresada és que, en els sistemes en temps real, cal definir sempre una única font de veritat.

Incidència 4: Accions concurrents. Es van detectar condicions de cursa en accions que es podien produir simultàniament, com una doble actualització de la puntuació ELO en finalitzar una partida. Es van resoldre amb operacions de base de dades atòmiques. La lliçó apresada és que no n'hi ha prou de comprovar una condició abans d'actuar si l'operació en conjunt no és atòmica.

Incidència 5: Seguretat de les credencials. Es va detectar que algunes credencials estaven escrites directament al codi del repositori. Es van extreure a variables d'entorn i a secrets, i es van excloure del control de versions. La lliçó apresada és que la seguretat s'ha de tenir present des de l'inici del projecte i no com una tasca final.

En conjunt, la resolució d'aquestes incidències ha estat un dels aspectes més formatius del projecte. Han obligat a entendre en profunditat el funcionament de les tecnologies emprades i han reforçat la importància de prendre decisions tècniques ben fonamentades i de no ajornar les qüestions de qualitat i de seguretat.

3. Elements finals

3.1. Conclusions

Resum dels resultats

El projecte ChessHub ha assolit tots els objectius que es va proposar. El resultat és una aplicació web d'escacs completa, funcional i desplegada en un entorn de producció real, que permet jugar partides en temps real, entrenar contra una intel·ligència artificial, resoldre problemes tàctics i relacionar-se amb altres jugadors. La solució integra una arquitectura de tres capes, un servei de temps real i un motor d'escacs professional, i s'ha construït amb tecnologies actuals i àmpliament utilitzades a la indústria.

Més enllà del compliment dels objectius, el projecte ha estat capaç d'incorporar diverses ampliacions i de resoldre, pel camí, un seguit d'incidències tècniques de certa complexitat. El producte final no és un prototip, sinó una aplicació operativa que es podria fer servir de manera real.

Valoració personal

El desenvolupament de ChessHub ha estat una experiència molt enriquidora. M'ha permès aplicar, de manera integrada i sobre un producte real, els coneixements adquirits durant tot el cicle formatiu: el desenvolupament d'interfícies, la programació de la lògica de servidor, el disseny i la gestió de bases de dades i el desplegament d'aplicacions. El fet d'haver de fer encaixar totes aquestes peces en un únic projecte ha aportat una visió de conjunt que difícilment s'obté treballant cada àmbit per separat.

La part més exigent i, alhora, la més satisfactòria ha estat la comunicació en temps real i la integració del motor d'intel·ligència artificial, dos reptes que han obligat a aprofundir en tecnologies que no es treballen habitualment a classe. Treballar sobre un entorn de producció real, amb contenidors i orquestració, ha aportat una visió pràctica i valuosa del cicle de vida complet d'una aplicació, des del codi fins al desplegament i el manteniment. El balanç global de l'experiència és, doncs, molt positiu.

Propostes de millora

Tot i que el projecte compleix els objectius plantejats, s'hi identifiquen diversos marges de millora. Convindria ampliar la cobertura de les proves automatitzades per cobrir un nombre més gran de casos, refinar la interfície en els dispositius de pantalla més petita i millorar els missatges d'ajuda i d'error perquè orientin millor l'usuari en les situacions problemàtiques. Aquestes millores no afecten el funcionament actual, però en consolidarien la qualitat.

Línies futures de desenvolupament

De cara al futur, ChessHub es podria ampliar amb noves funcionalitats que enriquirien encara més la plataforma:

- Tornejos i competicions organitzades, amb emparellaments automàtics i classificacions pròpies.
- Una col·lecció de problemes més extensa i classificada per temàtica.
- Una anàlisi de partides assistida pel motor, amb una avaluació de cada jugada.
- La recuperació autònoma de contrasenyes, sense intervenció de l'administrador.
- Una aplicació mòbil nativa que complementés la versió web actual.

Aquestes línies confirmen que el projecte, més enllà del seu estat actual, té recorregut i marge de creixement.

3.2. Referències i bibliografia

Referències tècniques i recursos web

Per al desenvolupament del projecte s'ha consultat, principalment, la documentació oficial de les tecnologies emprades:

- Documentació oficial d'Angular: angular.dev
- Documentació oficial de CodeIgniter 4: codeigniter.com/user_guide
- Documentació oficial de Socket.IO: socket.io/docs
- Documentació oficial de Kubernetes: kubernetes.io/docs
- Documentació de MariaDB: mariadb.org/documentation
- Projecte Stockfish (motor d'escacs): stockfishchess.org
- MDN Web Docs, referència de les tecnologies web: developer.mozilla.org

Eines utilitzades

- Windsurf, com a editor de codi.
- Git i GitHub, per al control de versions i l'allotjament del repositori.
- Docker i Kubernetes, per a la contenidorització i el desplegament.
- Harbor, com a registre d'imatges de contenidors.
- PHPUnit i Vitest, per a les proves automatitzades.

Documentació del projecte

Aquesta memòria es complementa amb la resta de documentació elaborada per al projecte:

- Manual tècnic de ChessHub.
- Manual d'usuari de ChessHub.
- Pla de proves del projecte.

3.3. Annexos

Annex A. Diagrames

En el Github, concretament a l'apartat de **/docs**, es troben els diagrames d'arquitectura general del sistema i de entitat-relació de la base de dades:

<https://github.com/OriolTorraTudela/chesshub.git>

Annex B. Model de dades

La base de dades del sistema està formada per quinze taules relacionals. La taula següent en descriu breument el contingut:

Taula	Contingut
users	Comptes d'usuari i credencials d'accés.
profiles	Dades de perfil de cada usuari i puntuació ELO.
refresh_tokens	Testimonis de renovació de sessió.
themes	Temes visuals disponibles per al taulell.
elo_history	Evolució històrica de la puntuació ELO.
games	Partides disputades entre jugadors.
moves	Jugades de les partides entre jugadors.
bot_games	Partides disputades contra la intel·ligència artificial.
bot_moves	Jugades de les partides contra la IA.
game_analysis	Resultats de l'anàlisi de partides.
puzzles	Col·lecció de problemes tàctics d'escacs.
puzzle_attempts	Intents de resolució dels problemes.
reports	Informes i denúncies per a l'administració.
friendships	Relacions d'amistat entre usuaris.
direct_messages	Missatges privats entre amics.

Annex C. Documentació complementària

Per a una descripció tècnica detallada del sistema, instal·lació, configuració, model de dades, API, seguretat i manteniment, cal consultar el Manual tècnic de ChessHub. Per a la guia d'ús completa de l'aplicació, adreçada a l'usuari final, cal consultar el Manual d'usuari. El detall de les proves realitzades i de l'estratègia de verificació es recull al Pla de proves del projecte. Això, també es troba en l'apartat de **/docs** del Github:

<https://github.com/OriolTorraTudela/chesshub.git>